

Codex Enterprise Rollout Design

Linton Labs | Legacy billing platform first | Partner-ready recommendation

Recommendation: establish one governed default for the billing pilot, prove it with reviewable evidence, then expand one bounded slice at a time.

Explicit rollout decisions

1. Surface strategy. Billing engineers default to the Codex IDE extension for day-to-day edits; platform and SRE use the Codex app for reviews and runbooks; CLI is for scripted, reproducible tasks. Use cloud tasks only for explicitly approved, non-sensitive work. Use worktrees for parallel or risky changes; keep single-file fixes in the current branch.

2. Model strategy. Use the current approved general model by default; use the latest approved API-key workflow model for automation. Escalate reasoning only for high-impact design or debugging. Spark is limited to low-latency, low-risk loops with tighter scope and explicit validation.

3. Context continuity. Checkpoint at task boundaries, before compaction, and before switching surfaces. Preserve decisions, file scope, commands, tests, and open risks in the task record. Resume from the last checkpoint rather than guessing.

4. Governance bundle. Ship a versioned config.toml, repo AGENTS.md, and .rules allowlist. AGENTS.md defines verification commands, review guidance, and the done definition; rules block risky commands and require human approval for release-affecting actions.

Safety and operating controls

5. Cloud and secrets. Cloud environments are off by default for billing work. Setup scripts are pinned and cached; secrets stay in the approved secret store and are never written to prompts, logs, or repositories. Internet access is disabled unless an allowlisted dependency is approved.

6. Review and integrations. Start GitHub review manually with @codex review; enable automatic reviews only after evidence quality and reviewer burden meet the pilot scorecard. Slack and Linear may receive summaries and bounded follow-ups, never merge, release, or permission decisions.

7. Admin and monitoring. Enable local and cloud surfaces through RBAC groups. Track analytics for adoption and workflow health; track compliance logs for approvals, sandbox mode, environment, access, and review disposition. The support runbook begins with approvals, sandbox, environment selection, and entitlements.

Windows posture. Prefer native Windows sandbox mode unelevated. Use elevated only when prerequisites are documented and approved; use WSL semantics when the repo requires Linux tooling. If elevated setup is blocked, fall back to the approved unelevated path and record the limitation.

Guardrails

Scope: billing repo and approved adjacent files only; no production testing, release, merge, or permission changes. **Evidence:** every run records scope, model, surface, commands, tests, approvals, and reviewer disposition. **Human gates:** humans accept findings, approve patches, merge, release, and expand scope.

Recovery patterns

Long-running task: checkpoint at each milestone; if context is compacted, reload the checkpoint and confirm scope, decisions, tests, and open risks before continuing. **Failure or drift:** stop, capture the last known-good state, revert or isolate the change, switch to a safer surface if needed, rerun focused verification, and request reviewer disposition before resuming.

Pilot exit criteria: reviewers can reproduce the workflow, approve or reject evidence, and recover from interruption; adoption and compliance signals are visible; no release or access decision is delegated. Expand to one adjacent repo or team only after the billing pilot meets these criteria.